



Query Data with DynamoDB

M

Mohammed Dawoud Mota

```
CloudShell

us-east-1 +

{"Id": {
  "N": "201"
},
"URL": {
  "S": "https://youtu.be/SzvdEf9y0eA"
}
}

~ $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"101"}}' \
> --consistent-read \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 1.0
  }
}

~ $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"202"}}' \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "Item": {
    "ContentType": {
      "S": "Video"
    },
    "Title": {
      "S": "Don't miss out!"
    }
  },
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 0.5
  }
}

~ $
```



Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed NoSQL database service that lets me store and retrieve data quickly without having to worry about servers, scaling, or maintenance. It's useful because it handles huge amounts of traffic automatically, gives fast performance at any scale, and lets me design flexible data models using partition keys and sort keys. It also supports features like transactions, fine-grained access control, and on-demand scaling, which makes it a strong choice for applications that need reliability and speed without the overhead of managing a traditional database.

How I used Amazon DynamoDB in this project

In today's project, I used Amazon DynamoDB to store and work with the data for my Forum, Post, and Comment tables. I created the tables, added items to them, and then practiced running different kinds of queries using both the AWS Console and CloudShell. I queried items by their partition keys, used projection expressions to return only specific attributes, and even ran a transaction that added a new comment while updating the comment count in the Forum table at the same time. Overall, I used DynamoDB to model relationships, retrieve data efficiently, and keep everything consistent across the tables.



Mohammed Dawoud M...

NextWork Student

nextwork.org

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how strict DynamoDB is about using the partition key for almost every meaningful query. I knew NoSQL databases worked differently from relational ones, but actually running the commands made it clear that if your data model isn't planned well, certain queries just aren't possible. It was a good reminder that DynamoDB rewards thoughtful design up front, because the database won't bend to fix a bad structure later.

This project took me...

This project took me 30 minutes to complete.



Querying DynamoDB Tables

A partition key is the main attribute DynamoDB uses to organize and locate data, acting as the first filter that determines which group of items your query will search through. It's essential for structuring how data is stored and retrieved, and in this project's tables it uniquely identifies each item, which makes finding specific data fast and efficient.

A sort key is a secondary attribute that DynamoDB uses to further filter and organize items after the partition key has been applied, letting you narrow down results within the same partition. It's especially useful when multiple items share the same partition key, because the combination of the partition key and sort key creates a unique primary key that allows you to query specific items or ranges of items efficiently.



▼ Scan or query items

☐ Scan

☒ Query

Select a table or index

Table - Comment ▼

Select attribute projection

All attributes ▼

Partition key

Attribute

Id

Value

I have a question/Just Complete Project #7 Depend

Sort key - optional

Attribute

CommentDateTime

Value

Greater than ▼

2024-09-01

☐ Sort descending

► Filters - optional

Run

Reset



Limits of Using DynamoDB

My query resulted in an error because DynamoDB requires me to use the partition key whenever I run a query, and I tried to filter only by the PostedBy attribute without providing the Id value first. Since queries must always start by specifying the partition key, DynamoDB couldn't process the request, which shows why good data modeling is so important when designing tables and planning the types of queries I'll need later.

From the Comment table, I can easily extract insights like which comments belong to a specific post, who posted each comment, and when those comments were made, because the table is structured around the post ID as the partition key and the comment timestamp as the sort key. However, I can't easily extract insights that require filtering without the partition key, such as finding all comments made by a specific user across every post, because DynamoDB queries always require the partition key first. This limitation shows why data modeling matters so much in DynamoDB and why some types of analysis are harder without a relational structure.



▼ Scan or query items

☐ Scan

☒ Query

Select a table or index

Table - Comment ▼

Select attribute projection

All attributes ▼

Partition key

Attribute

Id

Value

Enter attribute value

⛔ The partition key filter cannot be empty.

Sort key - optional

Attribute

CommentDateTime

Value

Equal to ▼

Enter attribute value

☐ Sort descending

▼ Filters - optional

Attribute name	Condition	Type	Value	
Posted	Equal to ▼	String ▼	User Abdulrahm	Remove



Running Queries with CLI

The last query I ran in CloudShell was a get-item command on the ContentCatalog table using the Id value 202. I used a projection expression to return only the Title, ContentType, and Services attributes, and I also included the return-consumed-capacity option so I could see how many read units the request used. The exact command I ran was `aws dynamodb get-item --table-name ContentCatalog --key '{"Id":{"N":"202"}}' --projection-expression "Title, ContentType, Services" --return-consumed-capacity TOTAL`, and it returned the selected fields for that item along with the consumed capacity details.

The three query options each change how DynamoDB returns the data. Consistent-read tells DynamoDB to give me the most up-to-date version of the item instead of the eventually consistent version it normally returns. Projection-expression lets me choose only the specific attributes I want back, which keeps the response smaller and more efficient. Return-consumed-capacity shows me how much read capacity the request used, which helps me understand the cost and performance impact of the query.



```
CloudShell

us-east-1 +

{"Id": {
  "N": "201"
},
"URL": {
  "S": "https://youtu.be/SzvdEf9y0eA"
}
}

~ $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"101"}}' \
> --consistent-read \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 1.0
  }
}

~ $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"202"}}' \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "Item": {
    "ContentType": {
      "S": "Video"
    },
    "Title": {
      "S": "Don't miss out!"
    }
  },
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 0.5
  }
}

~ $
```



Transactions

A transaction is a way to group multiple database operations together so they either all succeed or none of them go through, which keeps the data consistent across related tables. In this project, I used a transaction to add a new comment to the Comment table and update the comment count in the Forum table at the same time, making sure both changes happened together without any risk of one updating and the other failing.

I ran the transaction by sending DynamoDB a single request that bundled two operations together so they would either both succeed or both fail. In this case, I added a new comment to the Comment table and updated the comment count in the Forum table at the same time. I did it using a `transact-write-items` command in CloudShell, where I defined both actions inside one JSON payload. DynamoDB processed them as one atomic operation, which guaranteed that the data stayed consistent across both tables.



```
CloudShell
us-east-1 +
~ $ aws dynamodb transact-write-items --client-request-token TRANSACTION1 --transact-items '[
> {
>   "Put": {
>     "TableName": "Comment",
>     "Item": {
>       "Id": {"S": "Events/Do a Project Together - NextWork Study Session"},
>       "CommentDateTime": {"S": "2024-9-27T17:47:30Z"},
>       "Comment": {"S": "Excited to attend!"},
>       "PostedBy": {"S": "User Connor"}
>     }
>   },
>   {
>     "Update": {
>       "TableName": "Forum",
>       "Key": {"Name": {"S": "Events"}},
>       "UpdateExpression": "ADD Comments :inc",
>       "ExpressionAttributeValues": { ":inc": {"N": "1"} }
>     }
>   }
> ]'
~ $
```